

Agent Memory Selection under Bounded Context

Which History Records Matter for Software-Engineering Agents?

Jorge Simão

Draft — September 2026

Abstract

Long-running language-model agents accumulate task statements, model actions, tool observations, source-code views, edits, tests, error messages, and intermediate hypotheses. The common baseline is to replay as much recent history as the model context permits, but recency alone ignores the causal and semantic roles of agent state. This paper studies agent-history selection as a separate problem from inference-engine caching: given a canonical trajectory, which records should remain active for the next model call?

We introduce an engine-independent record abstraction and evaluate bounded memory policies for software-engineering agents. The first harness is mini-swe-agent, whose linear message history, bash-only action interface, and minimal control loop make memory selection directly observable. We compare full history, token and turn tails, independently controlled head and tail turns, causal-bundle recency, a typed progress spine, lexical and dense retrieval over prior records, hybrid policies, and oracle utility selection. We separate record selection from within-record materialization and evaluate both frozen-trajectory counterfactuals and autonomous SWE-bench runs.

The primary outcomes are official task resolution, model calls, selected input tokens, repeated source acquisition, repeated verification, and first-divergence behavior. We additionally measure the age and semantic type of historical records that remain causally useful. The goal is not to optimize one serving engine, but to derive a portable agent-memory policy that can later be realized as text, persistent native K/V, or other backend state. We hypothesize that typed causal retention preserves agent quality more efficiently than token recency, and that its first benefit appears as equal task success with fewer calls, rereads, and retained tokens.

1 Introduction

Software-engineering agents differ from ordinary retrieval-augmented prompts in one crucial way: their context is not a static evidence set. It is a growing history of actions and consequences. A coding trajectory may contain the original task, source-code observations, edits, failed commands, test results, intermediate hypotheses, and final verification. Many of these records become irrelevant quickly, while others remain necessary long after they were created.

The usual history policy is recency. Given a context limit, an agent keeps the latest messages or tokens and discards older state. This is simple but ignores causal structure. An action and its observation form a unit. A source view may remain important after several later path-discovery commands. A mutation can remain relevant until a subsequent verification. A test result may be obsolete after another edit, while the task statement remains permanently relevant.

This paper isolates the memory-policy problem from the serving-engine problem. At model turn t , let the full canonical history be

$$H_t = [r_1, r_2, \dots, r_n],$$

where each r_i is a logical agent record. A memory policy computes

$$S_t = \pi(H_t, Q_t, B),$$

where Q_t is the active task/state and B is a context budget. The selected plan S_t is then serialized to an ordinary model request. No native K/V mechanism is required for the main experiments.

This separation is deliberate. Prior PRA work studies retrieval, native-memory materialization, cross-document composition, and engine-specific K/V reuse. Those systems questions should not determine which semantic records an agent needs. Here we first identify useful logical memory policies. A separate runtime paper can later ask whether different engines realize the same selected plan correctly and efficiently.

We begin with mini-swe-agent because its control loop is minimal: a linear message history is sent to the model, the model emits shell actions, the environment executes them, and observations are appended to the same history. This minimizes hidden harness state and makes memory intervention easy to audit.

Research questions. We ask:

1. How much agent history can be removed before official task quality degrades?
2. Are causal-bundle and semantic-role policies more efficient than token or turn recency?
3. Which record types—source views, mutations, verification, progress, or task state—have the longest useful lifetimes?
4. Does bounded memory increase recovery behaviors such as rereading source or repeating tests?
5. How well do lexical, dense, and hybrid record retrieval approximate an oracle future-utility policy?
6. Do the best policies transfer to another agent harness and to persistent-native-memory execution?

Contributions planned. We target five contributions:

1. An engine-independent typed record model for agent trajectories.
2. A controlled policy ladder from token recency to causal, head–middle–tail, and role-aware retention.
3. A two-stage evaluation combining frozen-trajectory counterfactuals with autonomous SWE-bench runs.
4. Agent-specific efficiency metrics including avoidable rereads, repeated verification, and memory dependency horizon.
5. A frozen set of qualified logical policies that can later be realized by native-K/V runtimes without retuning the semantics.

2 Background and Motivation

2.1 Agent history differs from document retrieval

Document retrieval typically selects evidence that is externally stored and largely immutable. Agent memory contains generated and observed state whose meaning depends on the trajectory. A source observation may explain a later edit; an edit invalidates earlier verification; a rejected action may produce an error observation without becoming part of the accepted action history.

These dependencies motivate a record graph rather than an undifferentiated token stream.

2.2 Why mini-swe-agent

mini-swe-agent maintains a linear message list. At each step the model receives that list, returns one or more actions, the environment executes those actions, and formatted observations are appended. Because there is little additional memory management in the harness, the trajectory itself is the natural object for intervention.

We use mini-swe-agent as the first experimental harness, then test whether the highest-performing policies transfer to a richer open-source agent and to a record-native PRA agent.

mini-swe-agent exposes one generic Bash action interface. Consequently, tool choice is not a meaningful routing variable in this first study: the scientific variables are which action-observation history survives and how large tool observations are represented. We do not place a PRA gateway in the primary factorial. Gateway tool routing and result compaction are separate interventions that can be tested only after the logical memory policy is fixed.

3 Agent Record Model

We normalize trajectory messages into typed records:

$$\mathcal{R} = \{\text{TASK}, \text{ACTION}, \text{OBSERVATION}, \text{SOURCE}, \text{MUTATION}, \\ \text{VERIFICATION}, \text{PROGRESS}, \text{ERROR}, \text{FINALIZATION}\}.$$

Each record contains a stable identifier, turn index, semantic type, causal group, source identity/spans where applicable, mutation target, and logical creation time.

3.1 Causal bundles

Actions and observations are grouped:

$$\text{ACTION}_i \leftrightarrow \text{OBSERVATION}_i.$$

Mutations and later verification can form a higher-level dependency:

$$\text{MUTATION}_i \rightarrow \text{VERIFICATION}_j.$$

Policies may rank individual records, but required bundles are rounded up unless the experiment explicitly tests bundle-breaking.

4 Memory Selection Policies

4.1 Full history

FULL retains all canonical records and defines the reference trajectory.

4.2 Recency baselines

We test:

- **TOKEN-TAIL**: last B tokens;
- **TURN-TAIL**: last N complete turns;
- **RECORD-TAIL**: last M records;
- **CAUSAL-TAIL**: last N complete causal bundles.

4.3 Independent head, middle, and tail

Recency and early-run retention are separate variables. We therefore define a three-zone policy

$$S_t = I \cup H_h \cup \pi_{\text{middle}}(M_t, Q_t, B') \cup T_\tau,$$

where I is the immutable system prompt and task statement, H_h contains the first h complete action–observation turns, T_τ contains the latest τ complete turns, and the selector operates only on the non-overlapping middle M_t . The head and tail controls are independent; $h = 0$ never removes I . Our pilot grid uses

$$h \in \{0, 1, 2, 4\}, \quad \tau \in \{1, 2, 3, 5, 10\}.$$

Frozen replay eliminates dominated combinations before autonomous evaluation. This factorization tests whether early localization or plan formation remains useful after leaving a conventional recency window.

Head and tail turns remain causally complete. An oversized tool observation is the only permitted within-turn exception: its action, return code, resource identity, mutation/verification status, error headline, and causal identity remain present while its body may use a separately recorded detail policy. When immutable/head/tail state exceeds the requested budget, we report mandatory overflow and realized retention rather than silently breaking a turn.

4.4 Typed progress spine

The structured policy retains

$$S_{\text{spine}} = T \cup R_{\text{recent}} \cup S_{\text{source}} \cup M_{\text{mutation}} \cup V_{\text{verification}} \cup P_{\text{progress}},$$

where each component denotes the latest required records of that semantic role.

4.5 Retrieval policies

We compare lexical, dense, and hybrid retrieval over historical records using a query formed from

$$\text{task} + \text{current progress} + \text{latest active observation}.$$

The hybrid policy combines a mandatory structured spine with retrieved records.

4.6 Resource/effect DAG and negative exclusion

Positive selection asks which records are likely useful. We additionally test the dual question: which records are operationally obsolete under a declared resource-state abstraction? We construct an acyclic graph over actions, observations, immutable resource versions, and precisely defined projections. Typed edges include RETURNS, READSFROM, WRITES, SUPERSEDEDBY, and SUBSUMEDBY. Effects carry provenance from declared tool semantics, runtime tracing, static heuristics, or UNKNOWN; unknown effects prohibit certified exclusion.

The two-stage policy is

$$H_t \xrightarrow{\text{certified exclusion}} H'_t \xrightarrow[\text{only if still over } B]{\pi_{\text{positive}}} S_t.$$

A certificate names the rule, witnesses, proof scope, resource-version fingerprints, cwd/environment identity, completeness conditions, and explicit assumptions. We separate three tiers: DAG-CERTIFIED, DAG-RECOVERABLE, and DAG-HEURISTIC. Only the first is removed by the conservative $B = 100\%$ exclusion arm; the others require the same counterfactual and autonomous qualification as positive retrieval.

The scope of “certified” is deliberately narrow. A later record can be proven to supersede an older current-state projection, but this does not prove that the older text contains no decision-relevant diagnostic fact. Moreover, deleting even byte-identical repeated text changes token positions and attention. Thus the claimed property is *certificate-backed operational obsolescence*, not risk-free behavioral equivalence. Behavioral preservation remains an empirical outcome measured by frozen replay and autonomous tasks.

Initial rules fail closed: system/task/current state and unresolved errors are protected; arbitrary or untraced Bash is UNKNOWN; output must be complete and non-truncated; file reads require canonical paths and version/span identity; git queries require separate HEAD, index, and worktree identities; and action–observation bundles remain intact. A write invalidates an old record as an authority on current state but does not by itself authorize semantic deletion. Similarly, a later passing test does not automatically erase a prior failure’s diagnostic information.

4.6.1 Tool-semantics ownership and runtime handoff

mini-swe-agent exposes only one generic Bash tool, so the Paper 8.5 harness provides a Bash-specific semantics adapter that classifies common commands and, when available, augments them with runtime filesystem/version tracing. This special case belongs in the harness-facing adapter, not in a PRA inference engine. Arbitrary shell constructs remain unknown unless traced.

The portable interface is instead a tool-semantics provider. A developer or agent SDK can declare generic reads, writes, deletes, verification targets, resource versions, causal dependencies, output completeness, and confidence. It may also attach record directives such as protected, certified-excludable, or preferred-for-selection, with a certificate and witness IDs. Future tool categories implement the same effect vocabulary without teaching the engine about Bash, Excel, browsers, databases, or particular agent frameworks.

When the policy returns to Paper 4.5, the PRA runtime consumes only a frozen logical plan and generic record metadata. It validates identities, dependencies, and lifecycle constraints and realizes the chosen records physically; it does not infer application-specific semantics or rerun the DAG policy inside each engine. This preserves one agent-independent engine contract while allowing a semantically richer harness or SDK to make selection decisions.

4.7 Oracle

A retrospective oracle estimates future usefulness through record/bundle ablation on successful trajectories. It is an upper bound and is never used as a deployment policy.

5 Materialization Policies

Record selection and record detail are separate variables.

For selected records, we compare:

- full record;
- matched source span;
- bounded head/tail;
- hierarchical child span;
- optional summary.

The first experiments use whole-record materialization so that selector effects remain identifiable. A targeted second factorial applies detail reduction only to oversized tool observations. Source views retain matched symbol/file spans; test outputs retain the command, return code, failed-test identities, exception, and relevant stack frames; search results retain matched lines and locations; mutations retain their changed span or diff. This reuses gateway-style compaction semantics without introducing a gateway or an engine into the study.

6 Experimental Design

6.1 Phase I: frozen-trajectory screening

We begin from successful full-history trajectories. At each historical model call we restore the corresponding workspace, apply a candidate memory policy, and query the same frozen model.

We measure:

- exact next-response agreement;
- exact action/command agreement;
- action equivalence;
- target-file agreement;
- first-divergence turn;
- retained tokens and records.

Frozen screening permits broad policy and budget sweeps without paying for full autonomous reruns.

The repository state is part of the frozen decision point. Replaying turn t restores the exact pre-action workspace, not merely the textual history, so a candidate command is evaluated against the same files as the reference action.

6.2 Phase II: autonomous evaluation

Policies that survive frozen screening are run autonomously from a fresh task sandbox. Final success is measured by the official SWE-bench grader.

Autonomous evaluation is required because a divergent action may still recover later, while a seemingly small memory omission may cause many additional calls.

6.3 Benchmarks

We begin with the existing SWE-bench Verified Easy-14 task sequence, then expand to Easy-50 and a broader Verified sample.

Easy-14 is a mechanism/debug cohort. Publication-level claims require a larger task set.

6.4 Budgets

We evaluate relative budgets

100, 90, 75, 50, 25%

and absolute budgets

2K, 4K, 8K, 16K, 32K.

Every request records both requested and realized retention.

7 Metrics

7.1 Task outcome

Primary:

SWE-bench resolved.

Additional metrics are model calls to completion, shell actions, and final patch/test status.

7.2 Memory efficiency

We record full-history tokens, selected tokens, retention fraction, selected record count, selected records by semantic role, and the largest retained record.

For DAG policies we additionally report certified, recoverable, heuristic, and unknown-effect tokens separately; certificate coverage and abstention; records removed per rule; and certificate violations under frozen add-back tests. A certificate can be operationally sound while the LLM still changes behavior, so operational certificate validity and next-action survival are distinct metrics.

7.3 Recovery behavior

A memory policy may preserve final task success by forcing the agent to reacquire forgotten state. We therefore count repeated source reads, overlapping source rereads, repeated searches for the same fact, repeated tests without relevant intervening mutation, and repeated failed edits. We call this *reacquisition overhead*.

7.4 Dependency horizon

For a useful historical record r_i consumed at turn t , define

$$d(r_i, t) = t - \text{created}(r_i).$$

We report dependency-age distributions by record type. This estimates how long coding-agent memory must remain accessible.

8 Planned Policy Matrix

Table 1: Primary selector-by-budget matrix. Each cell reports official task success, calls/task, selected tokens, and avoidable rereads.

Policy	100%	90%	75%	50%	25%
FULL	—	—	—	—	—
Token tail	TBD	TBD	TBD	TBD	TBD
Turn tail	TBD	TBD	TBD	TBD	TBD
Causal tail	TBD	TBD	TBD	TBD	TBD
Progress spine	TBD	TBD	TBD	TBD	TBD
Lexical	TBD	TBD	TBD	TBD	TBD
Dense	TBD	TBD	TBD	TBD	TBD
Hybrid	TBD	TBD	TBD	TBD	TBD
DAG-certified exclusion	TBD	TBD	TBD	TBD	TBD
DAG-certified + hybrid	TBD	TBD	TBD	TBD	TBD
Oracle	TBD	TBD	TBD	TBD	TBD

Within each structured-policy row, we first screen the independent head–tail grid. The selected middle policy is then compared at matched materialized token counts. This prevents a large mandatory head from being mistaken for superior middle retrieval and prevents percentage budgets from hiding mandatory round-up.

8.1 Pilot structural screen

Before issuing counterfactual model calls, we ran the recorder and selector over three successful mini-swe-agent reference trajectories containing 69 historical model decisions. The full grid contains five budget fractions, four head floors, five tail floors, five initial middle policies, and two DAG-first compositions, for 48,300 selection decisions and 2,100 aggregated cells. This screen uses a whitespace counter and is therefore evidence only about structural feasibility, not model token counts or policy quality.

Table 2 shows one illustrative slice with one first turn and two latest turns retained. Whole causal bundles leave some budget unused. Conversely, immutable and mandatory state can exceed the ceiling. At a 90% request the basic head–tail policy overflows on 16 of 69 decisions, while the role-aware progress spine overflows on 22 of 69. These are expected round-up outcomes, concentrated when the trajectory is short; they demonstrate why requested percentage, realized retention, and mandatory overflow must be reported separately.

The legacy trajectories lack cwd, environment, output-completeness, and resource-version fingerprints. The certificate-backed DAG arm therefore correctly abstains and retains 100% at $B = 100\%$. Static Bash analysis exposes 458 superseded-current-state and 75 write-invalidated-state

candidate occurrences across the 69 growing prefixes, but no trace-exact duplicate and no certified exclusion. These are repeated candidate occurrences, not unique records, and are not removed. This is an instrumentation requirement, not a negative quality result.

Table 2: Aggregate realized retention in the structural-only pilot for $h = 1, \tau = 2$. These values do not measure next-action or task quality.

Policy	90%	75%	50%	25%
Middle none	55.5%	55.5%	55.5%	55.5%
Progress spine	66.5%	66.5%	66.5%	66.5%
Middle recency	85.9%	76.7%	59.6%	55.5%
Middle lexical	89.5%	77.1%	59.6%	55.5%
Spine + lexical	91.6%	80.0%	67.4%	66.5%

9 Semantic-Role Ablations

For the typed progress spine, we remove one role at a time:

–SOURCE, –MUTATION, –VERIFICATION, –PROGRESS, –RECENT.

We measure official success, calls, rereads, repeated tests, and first-divergence rate. These ablations test which semantic roles carry long-lived task state.

10 Oracle Utility

For a causal bundle g and future decision point t , define

$$U(g, t) = \text{Quality}(H_t) - \text{Quality}(H_t \setminus g).$$

The first implementation uses action-level behavioral outcomes rather than token-level attribution. Its future reference action is available only to the offline scorer and is never serialized into model input.

Bundles are analyzed in the hierarchy

record type $\rightarrow$ causal bundle $\rightarrow$ source span.

The oracle provides an upper bound on how compact agent history could be if future utility were known.

We distinguish two oracle levels. The *next-action oracle* fixes system/task/head/tail state and searches subsets of middle causal bundles, scoring exact command, semantic command equivalence, target file, and edit intent. Leave-one-bundle-out and role ablations provide the first utility labels; pairwise ablations and bounded beam or branch-and-bound search address non-additive bundle interactions before a subset is called minimal. The more expensive *rollout oracle* resumes a small predeclared set of decision points from exact workspace snapshots and scores eventual official success, calls, and recovery behavior. It tests whether next-action agreement is an adequate proxy for task utility.

We also express the oracle in the dual exclusion form. Holding protected state fixed, it searches for the largest set of middle bundles whose removal preserves the scoring outcome. This upper bound is compared with DAG-certified exclusions using behavioral precision (how often a certified

exclusion survives the counterfactual), removal recall relative to the oracle-removable set, and token coverage. “Never used again” in the observed path is not an oracle label; utility must be established by counterfactual generation or rollout.

11 Transfer

11.1 Second open-source harness

After policy qualification on mini-swe-agent, we transfer only the top policies to one richer harness such as SWE-agent or OpenHands. The goal is not a second exhaustive grid, but to test whether semantic-role findings generalize when the harness has richer tools and memory processing.

11.2 PRA Agent

The final transfer uses PRA Agent, where typed task, tool, result, progress, and subagent records are native objects. This tests whether a record-native agent can exploit the same policy more cleanly than a retrofitted linear-message harness.

12 Handoff to Native Runtime Evaluation

Paper 8.5 should freeze a small set of logical policies, for example

`{FULL, BALANCED, AGGRESSIVE}`.

A runtime study can then map the exact same `AgentMemoryPlan` to text replay, HF cache views, MLX interval views, vLLM pages, llama.cpp resident slots, and SGLang cache objects. The runtime study must not retune selection semantics per engine.

Paper 8.5 reports selected logical tokens, not K/V savings. K/V residence, copying, re-encoding, TTFT, and latency are measured only after the frozen plan is transferred to the runtime study.

13 Expected Outcomes

Our main hypothesis is

token recency < causal recency < typed progress spine + retrieval

at matched context budgets.

A positive result need not initially increase raw task success. A practically important result is same task success + fewer model calls + fewer rereads + fewer repeated tests + less retained history.

A negative result would also be informative: if simple recency performs as well as structured policies, coding-agent memory may be more local than assumed.

14 Limitations

Text realization recontextualizes selected records and therefore does not establish native-K/V equivalence. Frozen-trajectory screening also cannot replace autonomous evaluation. SWE-bench

coding tasks may not represent research, browser, or multimodal agents. Semantic record labels may depend on heuristics during the first implementation.

These limitations are deliberate. This paper isolates logical memory selection before introducing engine-specific physical state.

15 Conclusion

Agent memory selection is a distinct problem from transformer cache reuse. A serving engine may perfectly preserve selected native K/V while an agent policy selects the wrong history, and a good logical policy may be obscured by an incorrect engine implementation. We therefore study memory selection first, using an engine-independent record interface and a minimal software-engineering agent. The resulting policy frontier should provide a semantic contract for later native-memory runtimes: engines realize a frozen memory plan; they do not decide what the agent ought to remember.

A Artifact Schema

Each request/run should persist:

- task ID and repository revision;
- mini-swe-agent revision;
- model/tokenizer revision;
- generation settings;
- canonical trajectory digest;
- typed record list and causal groups;
- selector name/revision;
- materializer name/revision;
- requested/realized budget;
- selected record IDs/spans;
- selected token count;
- next model response/action;
- autonomous task result;
- reread/retest/recovery counters;
- failure taxonomy labels.

B Initial Failure Taxonomy

MEMORY_TASK_LOSS
MEMORY_SOURCE_LOSS
MEMORY_MUTATION_LOSS
MEMORY_VERIFICATION_LOSS
MEMORY_PROGRESS_LOSS
MEMORY_CAUSAL_PAIR_BREAK
MEMORY_STALE_HYPOTHESIS
MEMORY_REDUNDANT_REREAD
MEMORY_REDUNDANT_TEST
MEMORY_WRONG_FILE
MEMORY_REPEAT_FAILED_EDIT
MEMORY_FORMAT_REGRESSION
MEMORY_RECOVERY_SUCCESS
MEMORY_RECOVERY_FAILURE